

To use the `no_switch` await adapters, just change the type of that `co_await` expression from `IAsyncXxx` to `no_switch`, like this.

C++/WinRT

```
IAsyncAction async{ ProcessFeedAsync() };  
co_await static_cast<no_switch>(async);
```

Then, instead of looking for the three `await_xxx` functions that match `IAsyncXxx`, the C++ compiler looks for functions that match `no_switch`.

A deeper dive into `winrt::resume_foreground`

As of [C++/WinRT 2.0](#), the `winrt::resume_foreground` function suspends even if it's called from the dispatcher thread (in previous versions, it could introduce deadlocks in some scenarios because it only suspended if not already on the dispatcher thread).

The current behavior means that you can rely on stack unwinding and re-queuing taking place; and that's important for system stability, especially in low-level systems code. The last code listing in the section [Programming with thread affinity in mind](#), above, illustrates performing some complex calculation on a background thread, and then switching to the appropriate UI thread in order to update the user interface (UI).

Here's how `winrt::resume_foreground` looks internally.

C++/WinRT

```
auto resume_foreground(...) noexcept  
{  
    struct awaitable  
    {  
        bool await_ready() const  
        {  
            return false; // Queue without waiting.  
            // return m_dispatcher.HasThreadAccess(); // The C++/WinRT 1.0  
            implementation.  
        }  
        void await_resume() const {}  
        void await_suspend(coroutine_handle<> handle) const { ... }  
    };  
    return awaitable{ ... };  
};
```

This current, versus previous, behavior is analogous to the difference between `PostMessage` and `SendMessage` in Win32 application development. `PostMessage`

queues the work and then unwinds the stack without waiting for the work to complete. The stack-unwinding can be essential.

The `winrt::resume_foreground` function also initially only supported the [CoreDispatcher](#) (tied to a [CoreWindow](#)), which was introduced prior to Windows 10. We've since introduced a more flexible and efficient dispatcher: the [DispatcherQueue](#). You can create a [DispatcherQueue](#) for your own purposes. Consider this simple console application.

C++/WinRT

```
using namespace Windows::System;

winrt::fire_and_forget RunAsync(DispatcherQueue queue);

int main()
{
    auto controller{ DispatcherQueueController::CreateOnDedicatedThread() };
    RunAsync(controller.DispatcherQueue());
    getchar();
}
```

The example above creates a queue (contained within a controller) on a private thread, and then passes the controller to the coroutine. The coroutine can use the queue to await (suspend and resume) on the private thread. Another common use of [DispatcherQueue](#) is to create a queue on the current UI thread for a traditional desktop or Win32 app.

C++/WinRT

```
DispatcherQueueController CreateDispatcherQueueController()
{
    DispatcherQueueOptions options
    {
        sizeof(DispatcherQueueOptions),
        DQTYPE_THREAD_CURRENT,
        DQTAT_COM_STA
    };

    ABI::Windows::System::IDispatcherQueueController* ptr{};
    winrt::check_hresult(CreateDispatcherQueueController(options, &ptr));
    return { ptr, take_ownership_from_abi };
}
```

This illustrates how you can call and incorporate Win32 functions into your C++/WinRT projects, by simply calling the Win32-style [CreateDispatcherQueueController](#) function to create the controller, and then transfer ownership of the resulting queue controller to

the caller as a WinRT object. This is also precisely how you can support efficient and seamless queuing on your existing Petzold-style Win32 desktop application.

C++/WinRT

```
winrt::fire_and_forget RunAsync(DispatcherQueue queue);

int main()
{
    Window window;
    auto controller{ CreateDispatcherQueueController() };
    RunAsync(controller.DispatcherQueue());
    MSG message;

    while (GetMessage(&message, nullptr, 0, 0))
    {
        DispatchMessage(&message);
    }
}
```

Above, the simple **main** function begins by creating a window. You can imagine that this registers a window class, and calls **CreateWindow** to create the top-level desktop window. **CreateDispatcherQueueController** function is then called to create the queue controller before calling some coroutine with the dispatcher queue owned by this controller. A traditional message pump is then entered where resumption of the coroutine naturally occurs on this thread. Having done that, you can return to the elegant world of coroutines for your async or message-based workflow within your application.

C++/WinRT

```
winrt::fire_and_forget RunAsync(DispatcherQueue queue)
{
    ... // Begin on the calling thread...

    co_await winrt::resume_foreground(queue);

    ... // ...resume on the dispatcher thread.
}
```

The call to **winrt::resume_foreground** will always *queue*, and then unwind the stack. You can also optionally set the resumption priority.

C++/WinRT

```
winrt::fire_and_forget RunAsync(DispatcherQueue queue)
{
    ...
}
```

```

        co_await winrt::resume_foreground(queue, DispatcherQueuePriority::High);

        ...
    }

```

Or, using the default queuing order.

C++/WinRT

```

...
#include <winrt/Windows.System.h>
using namespace Windows::System;
...
winrt::fire_and_forget RunAsync(DispatcherQueue queue)
{
    ...

    co_await queue;

    ...
}

```

ⓘ Note

As shown above, be sure to include the projection header for the namespace of the type you're `co_await`-ing. For example, **Windows::UI::Core::CoreDispatcher**, **Windows::System::DispatcherQueue**, or **Microsoft::UI::Dispatching::DispatcherQueue**.

Or, in this case detecting queue shutdown, and handling it gracefully.

C++/WinRT

```

winrt::fire_and_forget RunAsync(DispatcherQueue queue)
{
    ...

    if (co_await queue)
    {
        ... // Resume on dispatcher thread.
    }
    else
    {
        ... // Still on calling thread.
    }
}

```

The `co_await` expression returns `true`, indicating that resumption will occur on the dispatcher thread. In other words, that queuing was successful. Conversely, it returns `false` to indicate that execution remains on the calling thread because the queue's controller is shutting down and is no longer serving queue requests.

So, you have a great deal of power at your fingertips when you combine C++/WinRT with coroutines; and especially when doing some old-school Petzold-style desktop application development.

Canceling an asynchronous operation, and cancellation callbacks

The Windows Runtime's features for asynchronous programming allow you to cancel an in-flight asynchronous action or operation. Here's an example that calls [StorageFolder::GetFilesAsync](#) to retrieve a potentially large collection of files, and it stores the resulting asynchronous operation object in a data member. The user has the option to cancel the operation.

C++/WinRT

```
// MainPage.xaml
...
<Button x:Name="workButton" Click="OnWork">Work</Button>
<Button x:Name="cancelButton" Click="OnCancel">Cancel</Button>
...

// MainPage.h
...
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Storage.Search.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Foundation::Collections;
using namespace Windows::Storage;
using namespace Windows::Storage::Search;
using namespace Windows::UI::Xaml;
...
struct MainPage : MainPageT<MainPage>
{
    MainPage()
    {
        InitializeComponent();
    }

    IAsyncAction OnWork(IInspectable /* sender */, RoutedEventArgs /* args
*/)
```

```

{
    workButton().Content(winrt::box_value(L"Working..."));

    // Enable the Pictures Library capability in the app manifest file.
    StorageFolder picturesLibrary{ KnownFolders::PicturesLibrary() };

    m_async =
picturesLibrary.GetFilesAsync(CommonFileQuery::OrderByDate, 0, 1000);

    IVectorView<StorageFile> filesInFolder{ co_await m_async };

    workButton().Content(box_value(L"Done!"));

    // Process the files in some way.
}

void OnCancel(IInspectable const& /* sender */, RoutedEventArgs const&
/* args */)
{
    if (m_async.Status() != AsyncStatus::Completed)
    {
        m_async.Cancel();
        workButton().Content(winrt::box_value(L"Canceled"));
    }
}

private:
    IAsyncOperation<::IVectorView<StorageFile>> m_async;
};
...

```

For the implementation side of cancellation, let's begin with a simple example.

C++/WinRT

```

// main.cpp
#include <iostream>
#include <winrt/Windows.Foundation.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace std::chrono_literals;

IAsyncAction ImplicitCancellationAsync()
{
    while (true)
    {
        std::cout << "ImplicitCancellationAsync: do some work for 1 second"
<< std::endl;
        co_await 1s;
    }
}

```

```

IAsyncAction MainCoroutineAsync()
{
    auto implicit_cancellation{ ImplicitCancellationAsync() };
    co_await 3s;
    implicit_cancellation.Cancel();
}

int main()
{
    winrt::init_apartment();
    MainCoroutineAsync().get();
}

```

If you run the example above, then you'll see **ImplicitCancellationAsync** print one message per second for three seconds, after which time it automatically terminates as a result of being canceled. This works because, on encountering a `co_await` expression, a coroutine checks whether it has been canceled. If it has, then it short-circuits out; and if it hasn't, then it suspends as normal.

Cancellation can, of course, happen while the coroutine is suspended. Only when the coroutine resumes, or hits another `co_await`, will it check for cancellation. The issue is one of potentially too-coarse-grained latency in responding to cancellation.

So, another option is to explicitly poll for cancellation from within your coroutine. Update the example above with the code in the listing below. In this new example, **ExplicitCancellationAsync** retrieves the object returned by the [winrt::get_cancellation_token](#) function, and uses it to periodically check whether the coroutine has been canceled. As long as it's not canceled, the coroutine loops indefinitely; once it is canceled, the loop and the function exit normally. The outcome is the same as the previous example, but here exiting happens explicitly, and under control.

C++/WinRT

```

IAsyncAction ExplicitCancellationAsync()
{
    auto cancellation_token{ co_await winrt::get_cancellation_token() };

    while (!cancellation_token())
    {
        std::cout << "ExplicitCancellationAsync: do some work for 1 second"
        << std::endl;
        co_await 1s;
    }
}

IAsyncAction MainCoroutineAsync()
{

```

```

    auto explicit_cancellation{ ExplicitCancellationAsync() };
    co_await 3s;
    explicit_cancellation.Cancel();
}
...

```

Waiting on `winrt::get_cancellation_token` retrieves a cancellation token with knowledge of the `IAsyncAction` that the coroutine is producing on your behalf. You can use the function call operator on that token to query the cancellation state—essentially polling for cancellation. If you're performing some compute-bound operation, or iterating through a large collection, then this is a reasonable technique.

Register a cancellation callback

The Windows Runtime's cancellation doesn't automatically flow to other asynchronous objects. But—introduced in version 10.0.17763.0 (Windows 10, version 1809) of the Windows SDK—you can register a cancellation callback. This is a pre-emptive hook by which cancellation can be propagated, and makes it possible to integrate with existing concurrency libraries.

In this next code example, `NestedCoroutineAsync` does the work, but it has no special cancellation logic in it. `CancellationPropagatorAsync` is essentially a wrapper on the nested coroutine; the wrapper forwards cancellation pre-emptively.

C++/WinRT

```

// main.cpp
#include <iostream>
#include <winrt/Windows.Foundation.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace std::chrono_literals;

IAsyncAction NestedCoroutineAsync()
{
    while (true)
    {
        std::cout << "NestedCoroutineAsync: do some work for 1 second" <<
std::endl;
        co_await 1s;
    }
}

IAsyncAction CancellationPropagatorAsync()
{
    auto cancellation_token{ co_await winrt::get_cancellation_token() };
    auto nested_coroutine{ NestedCoroutineAsync() };
}

```

```

        cancelation_token.callback([=]
        {
            nested_coroutine.Cancel();
        });

        co_await nested_coroutine;
    }

IAsyncAction MainCoroutineAsync()
{
    auto cancelation_propagator{ CancelationPropagatorAsync() };
    co_await 3s;
    cancelation_propagator.Cancel();
}

int main()
{
    winrt::init_apartment();
    MainCoroutineAsync().get();
}

```

CancelationPropagatorAsync registers a lambda function for its own cancellation callback, and then it awaits (it suspends) until the nested work completes. When or if **CancelationPropagatorAsync** is canceled, it propagates the cancellation to the nested coroutine. There's no need to poll for cancellation; nor is cancellation blocked indefinitely. This mechanism is flexible enough for you to use it to interop with a coroutine or concurrency library that knows nothing of C++/WinRT.

Reporting progress

If your coroutine returns either **IAsyncActionWithProgress**, or **IAsyncOperationWithProgress**, then you can retrieve the object returned by the **winrt::get_progress_token** function, and use it to report progress back to a progress handler. Here's a code example.

C++/WinRT

```

// main.cpp
#include <iostream>
#include <winrt/Windows.Foundation.h>

using namespace winrt;
using namespace Windows::Foundation;
using namespace std::chrono_literals;

IAsyncOperationWithProgress<double, double> CalcPiTo5DPs()
{
    auto progress{ co_await winrt::get_progress_token() };
}

```

```

        co_await 1s;
        double pi_so_far{ 3.1 };
        progress.set_result(pi_so_far);
        progress(0.2);

        co_await 1s;
        pi_so_far += 4.e-2;
        progress.set_result(pi_so_far);
        progress(0.4);

        co_await 1s;
        pi_so_far += 1.e-3;
        progress.set_result(pi_so_far);
        progress(0.6);

        co_await 1s;
        pi_so_far += 5.e-4;
        progress.set_result(pi_so_far);
        progress(0.8);

        co_await 1s;
        pi_so_far += 9.e-5;
        progress.set_result(pi_so_far);
        progress(1.0);

        co_return pi_so_far;
    }

    IAsyncAction DoMath()
    {
        auto async_op_with_progress{ CalcPiTo5DPs() };
        async_op_with_progress.Progress([](auto const& sender, double progress)
        {
            std::wcout << L"CalcPiTo5DPs() reports progress: " << progress <<
            L". "
                        << L"Value so far: " << sender.GetResults() << std::endl;
        });
        double pi{ co_await async_op_with_progress };
        std::wcout << L"CalcPiTo5DPs() is complete !" << std::endl;
        std::wcout << L"Pi is approx.: " << pi << std::endl;
    }

    int main()
    {
        winrt::init_apartment();
        DoMath().get();
    }

```

To report progress, invoke the progress token with the progress value as the argument. To set a provisional result, use the `set_result()` method on the progress token.

ⓘ Note

Reporting provisional results requires C++/WinRT version 2.0.210309.3 or later.

The above example chooses to set a provisional result for every progress report. You can choose to report provisional results any time, if at all. It need not be coupled with a progress report.

ⓘ Note

It's not correct to implement more than one *completion handler* for an asynchronous action or operation. You can have either a single delegate for its completed event, or you can `co_await` it. If you have both, then the second will fail. Either one of the following two kinds of completion handlers is appropriate; not both for the same async object.

C++/WinRT

```
auto async_op_with_progress{ CalcPiTo5DPs() };
async_op_with_progress.Completed([](auto const& sender, AsyncStatus /*
status */)
{
    double pi{ sender.GetResults() };
});
```

C++/WinRT

```
auto async_op_with_progress{ CalcPiTo5DPs() };
double pi{ co_await async_op_with_progress };
```

For more info about completion handlers, see [Delegate types for asynchronous actions and operations](#).

Fire and forget

Sometimes, you have a task that can be done concurrently with other work, and you don't need to wait for that task to complete (no other work depends on it), nor do you need it to return a value. In that case, you can fire off the task and forget it. You can do that by writing a coroutine whose return type is `winrt::fire_and_forget` (instead of one of the Windows Runtime asynchronous operation types, or `concurrency::task`).

C++/WinRT

```
// main.cpp
#include <winrt/Windows.Foundation.h>

using namespace winrt;
using namespace std::chrono_literals;

winrt::fire_and_forget CompleteInFiveSeconds()
{
    co_await 5s;
}

int main()
{
    winrt::init_apartment();
    CompleteInFiveSeconds();
    // Do other work here.
}
```

`winrt::fire_and_forget` is also useful as the return type of your event handler when you need to perform asynchronous operations in it. Here's an example (also see [Strong and weak references in C++/WinRT](#)).

C++/WinRT

```
winrt::fire_and_forget MyClass::MyMediaBinder_OnBinding(MediaBinder const&,
MediaBindingEventArgs args)
{
    auto lifetime{ get_strong() }; // Prevent *this* from prematurely being
    destructed.
    auto ensure_completion{ unique_deferral(args.GetDeferral()) }; // Take a
    deferral, and ensure that we complete it.

    auto file{ co_await
StorageFile::GetFileFromApplicationUriAsync(Uri(L"ms-
appx:///video_file.mp4")) };
    args.SetStorageFile(file);

    // The destructor of unique_deferral completes the deferral here.
}
```

The first argument (the *sender*) is left unnamed, because we never use it. For that reason we're safe to leave it as a reference. But observe that *args* is passed by value. See the [Parameter-passing](#) section above.

Awaiting a kernel handle

C++/WinRT provides a [winrt::resume_on_signal](#) function, which you can use to suspend until a kernel event is signaled. You're responsible for ensuring that the handle remains valid until your `co_await resume_on_signal(h)` returns. `resume_on_signal` itself can't do that for you, because you may have lost the handle even before the `resume_on_signal` starts, as in this first example.

C++/WinRT

```
IAsyncAction Async(HANDLE event)
{
    co_await DoWorkAsync();
    co_await resume_on_signal(event); // The incoming handle is not valid
    here.
}
```

The incoming **HANDLE** is valid only until the function returns, and this function (which is a coroutine) returns at the first suspension point (the first `co_await` in this case). While awaiting `DoWorkAsync`, control has returned to the caller, the calling frame has gone out of scope, and you no longer know whether the handle will be valid when your coroutine resumes.

Technically, our coroutine is receiving its parameters by value, as it should (see [Parameter-passing](#) above). But in this case we need to go a step further so that we're following the *spirit* of that guidance (rather than just the letter). We need to pass a strong reference (in other words, ownership) along with the handle. Here's how.

C++/WinRT

```
IAsyncAction Async(winrt::handle event)
{
    co_await DoWorkAsync();
    co_await resume_on_signal(event); // The incoming handle is valid
    here.
}
```

Passing a [winrt::handle](#) by value provides ownership semantics, which ensures that the kernel handle remains valid for the lifetime of the coroutine.

Here's how you might call that coroutine.

C++/WinRT

```
namespace
{
    winrt::handle duplicate(winrt::handle const& other, DWORD access)
    {
```

```

        winrt::handle result;
        if (other)
        {
            winrt::check_bool(::DuplicateHandle(::GetCurrentProcess(),
                other.get(), ::GetCurrentProcess(), result.put(), access,
FALSE, 0));
        }
        return result;
    }

    winrt::handle make_manual_reset_event(bool initialState = false)
    {
        winrt::handle event{ ::CreateEvent(nullptr, true, initialState,
nullptr) };
        winrt::check_bool(static_cast<bool>(event));
        return event;
    }
}

IAsyncAction SampleCaller()
{
    handle event{ make_manual_reset_event() };
    auto async{ Async(duplicate(event)) };

    ::SetEvent(event.get());
    event.close(); // Our handle is closed, but Async still has a valid
handle.

    co_await async; // Will wake up when *event* is signaled.
}

```

You can pass a timeout value to **resume_on_signal**, as in this example.

C++/WinRT

```

winrt::handle event = ...

if (co_await winrt::resume_on_signal(event.get(), std::literals::2s))
{
    puts("signaled");
}
else
{
    puts("timed out");
}

```

Asynchronous timeouts made easy

C++/WinRT is invested heavily in C++ coroutines. Their effect on writing concurrency code is transformational. This section discusses cases where details of asynchrony are

not important, and all you want is the result there and then. For that reason, C++/WinRT's implementation of the [IAsyncAction](#) Windows Runtime asynchronous operation interface has a [get](#) function, similar to that provided by `std::future`.

C++/WinRT

```
using namespace winrt::Windows::Foundation;
int main()
{
    IAsyncAction async = ...
    async.get();
    puts("Done!");
}
```

The **get** function blocks indefinitely, while the async object completes. Async objects tend to be very short-lived, so this is often all you need.

But there are cases where that's not sufficient, and you need to abandon the wait after some time has elapsed. Writing that code has always been possible, thanks to the building blocks provided by the Windows Runtime. But now C++/WinRT makes it a lot easier by providing the [wait_for](#) function. It's also implemented on [IAsyncAction](#), and again it's similar to that provided by `std::future`.

C++/WinRT

```
using namespace std::chrono_literals;
int main()
{
    IAsyncAction async = ...

    if (async.wait_for(5s) == AsyncStatus::Completed)
    {
        puts("done");
    }
}
```

ⓘ Note

`wait_for` uses `std::chrono::duration` at the interface, but it is limited to some range smaller than what `std::chrono::duration` provides (roughly 49.7 days).

The `wait_for` in this next example waits for around five seconds and then it checks completion. If the comparison is favorable, then you know that the async object completed successfully, and you're done. If you're waiting for some result, then you can simply follow that with a call to the [GetResults](#) method to retrieve the result.

❗ Note

wait_for and **get** are mutually exclusive (you can't call both of them). They each count as a *waiter*, and Windows Runtime asynchronous actions/operations support only a single waiter.

C++/WinRT

```
int main()
{
    IAsyncOperation<int> async = ...

    if (async.wait_for(5s) == AsyncStatus::Completed)
    {
        printf("result %d\n", async.GetResults());
    }
}
```

Because the async object has completed by then, the **GetResults** method returns the result immediately, without any further wait. As you can see, **wait_for** returns the state of the async object. So, you can use it for more fine-grained control, like this.

C++/WinRT

```
switch (async.wait_for(5s))
{
case AsyncStatus::Completed:
    printf("result %d\n", async.GetResults());
    break;
case AsyncStatus::Canceled:
    puts("canceled");
    break;
case AsyncStatus::Error:
    puts("failed");
    break;
case AsyncStatus::Started:
    puts("still running");
    break;
}
```

- Remember that **AsyncStatus::Completed** means that the async object completed successfully, and you may call the **GetResults** method to retrieve any result.
- **AsyncStatus::Canceled** means that the async object was canceled. A cancellation is typically requested by the caller, so it would be rare to handle this state. Typically, a canceled async object is simply discarded. You can call the **GetResults** method to rethrow the cancellation exception if you wish.

- **AsyncStatus::Error** means that the async object has failed in some way. You can call the **GetResults** method to rethrow the exception if you wish.
- **AsyncStatus::Started** means that the async object is still running. The Windows Runtime async pattern doesn't allow multiple waits, nor waiters. That means that you can't call **wait_for** in a loop. If the wait has effectively timed-out, then you're left with a few choices. You can abandon the object, or you can poll its status before calling the **GetResults** method to retrieve any result. But it's best just to discard the object at this point.

An alternative pattern is to check only for **Started**, and let **GetResults** deal with the other cases.

C++/WinRT

```
if (async.wait_for(5s) == AsyncStatus::Started)
{
    puts("timed out");
}
else
{
    // will throw appropriate exception if in canceled or error state
    auto results = async.GetResults();
}
```

Returning an array asynchronously

Below is an example of **MIDL 3.0** that produces *error MIDL2025: [msg]syntax error [context]: expecting > or, near "["*.

idl

```
Windows.Foundation.IAsyncOperation<Int32[]> RetrieveArrayAsync();
```

The reason is that it's invalid to use an array as a parameter type argument to a parameterized interface. So we need a less obvious way to achieve the aim of asynchronously passing an array back from a runtime class method.

You can return the array boxed into a **PropertyValue** object. The calling code then unboxes it. Here's a code example, which you can try out by adding the **SampleComponent** runtime class to a **Windows Runtime Component (C++/WinRT)** project, and then consuming that from (for example) a **Core App (C++/WinRT)** project.

C++/WinRT

```

// SampleComponent.idl
namespace MyComponentProject
{
    runtimeclass SampleComponent
    {
        Windows.Foundation.IAsyncOperation<IIInspectable>
RetrieveCollectionAsync();
    };
}

// SampleComponent.h
...
struct SampleComponent : SampleComponentT<SampleComponent>
{
    ...
    Windows::Foundation::IAsyncOperation<Windows::Foundation::IIInspectable>
RetrieveCollectionAsync()
    {
        co_return Windows::Foundation::PropertyValue::CreateInt32Array({ 99,
101 }); // Box an array into a PropertyValue.
    }
}
...

// SampleCoreApp.cpp
...
MyComponentProject::SampleComponent m_sample_component;
...
auto boxed_array{ co_await m_sample_component.RetrieveCollectionAsync() };
auto property_value{
boxed_array.as<winrt::Windows::Foundation::IPropertyValue>() };
winrt::com_array<int32_t> my_array;
property_value.GetInt32Array(my_array); // Unbox back into an array.
...

```

Important APIs

- [IAsyncAction](#) interface
- [IAsyncActionWithProgress<TProgress>](#) interface
- [IAsyncOperation<TResult>](#) interface
- [IAsyncOperationWithProgress<TResult, TProgress>](#) interface
- [SyndicationClient::RetrieveFeedAsync](#) method
- [winrt::fire_and_forget](#)
- [winrt::get_cancellation_token](#)
- [winrt::get_progress_token](#)
- [winrt::resume_foreground](#)

Related topics

- [Concurrency and asynchronous operations](#)
- [Handle events by using delegates in C++/WinRT](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

A completion source sample

Article • 06/24/2024

ⓘ Note

As an alternative to the sample code in this topic, there's the source code for a production-ready version of a task completion source implementation in the [cpp-async](#) [GitHub repo](#).

This topic demonstrates how you can author and consume your own completion source class, similar to .NET's [TaskCompletionSource](#).

Source code for the completion_source sample

The code in the listing below is offered as a sample. Its purpose is to illustrate how you could write your own version of it. For example, support for cancellation and error propagation are outside the scope of this sample.

C++/WinRT

```
#include <winrt/base.h>
#include <windows.h>

template <typename T>
struct completion_source
{
    completion_source()
    {
        m_signal.attach(::CreateEvent(nullptr, true, false, nullptr));
    }

    void set(T const& value)
    {
        m_value = value;
        ::SetEvent(m_signal.get());
    }

    bool await_ready() const noexcept
    {
        return ::WaitForSingleObject(m_signal.get(), 0) == 0;
    }

    void await_suspend(std::experimental::coroutine_handle<> resume)
    {
        m_wait.attach(winrt::check_pointer(::CreateThreadpoolWait(callback,
resume.address(), nullptr)));
    }
};
```

```

        ::SetThreadpoolWait(m_wait.get(), m_signal.get(), nullptr);
    }

    T await_resume() const noexcept
    {
        return m_value;
    }

private:

    static void __stdcall callback(PTP_CALLBACK_INSTANCE, void* context,
    PTP_WAIT, TP_WAIT_RESULT) noexcept
    {
        std::experimental::coroutine_handle<>::from_address(context)();
    }

    struct wait_traits
    {
        using type = PTP_WAIT;

        static void close(type value) noexcept
        {
            ::CloseThreadpoolWait(value);
        }

        static constexpr type invalid() noexcept
        {
            return nullptr;
        }
    };

    winrt::handle m_signal;
    winrt::handle_type<wait_traits> m_wait;
    T m_value{};
};

```

Offload completion to a separate coroutine

This section demonstrates one use case for **completion_source**. Create a new project in Visual Studio based on the **Windows Console Application (C++/WinRT)** project template, and paste the following code listing into `main.cpp` (expanding the definition of **completion_source** based on the listing in the previous section).

C++/WinRT

```

// main.cpp
#include "pch.h"

#include <winrt/base.h>
#include <windows.h>

```

```

template <typename T>
struct completion_source
{
    ... // Paste the listing of completion_source here.
}

using namespace std::literals;
using namespace winrt;
using namespace Windows::Foundation;

fire_and_forget CompleteAfterFiveSecondsAsync(completion_source<bool>&
completionSource)
{
    co_await 5s;
    completionSource.set(true);
}

IAsyncAction CompletionSourceExample1Async()
{
    completion_source<bool> completionSource;
    CompleteAfterFiveSecondsAsync(completionSource);
    co_await completionSource;
}

int main()
{
    auto asyncAction { CompletionSourceExample1Async() };
    puts("waiting");
    asyncAction.get();
    puts("done");
}

```

Encapsulate a completion_source in a class, and return a value

In this next example, a simple **App** class is used to encapsulate a **completion_source**, and return a value when it completes. Create a new project in Visual Studio based on the **Windows Console Application (C++/WinRT)** project template, and paste the following code listing into `main.cpp` (expanding the definition of **completion_source** based on the listing in the previous section).

C++/WinRT

```

// main.cpp
#include "pch.h"

#include <winrt/base.h>
#include <windows.h>

```

```

template <typename T>
struct completion_source
{
    ... // Paste the listing of completion_source here.
}

using namespace std::literals;
using namespace winrt;
using namespace Windows::Foundation;

struct App
{
    completion_source<winrt::hstring> m_completionSource;

    IAsyncOperation<winrt::hstring> CompletionSourceExample2Async()
    {
        co_return co_await m_completionSource;
    }

    winrt::fire_and_forget CompleteAfterFiveSecondsAsync()
    {
        co_await 5s;
        m_completionSource.set(L"Hello, World!");
    }
};

int main()
{
    App app;
    auto asyncAction{ app.CompletionSourceExample2Async() };
    app.CompleteAfterFiveSecondsAsync();
    puts("waiting");
    auto message = asyncAction.get();
    printf("%ls\n", message.c_str());
}

```

Related topics

- [Concurrency and asynchronous operations](#)
- [Advanced concurrency and asynchrony with C++/WinRT](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

XAML controls; bind to a C++/WinRT property

Article • 04/01/2023

A property that can be effectively bound to a XAML control is known as an *observable* property. This idea is based on the software design pattern known as the *observer pattern*. This topic shows how to implement observable properties in C++/WinRT, and how to bind XAML controls to them (for background info, see [Data binding](#)).

📘 Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with C++/WinRT, see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

What does *observable* mean for a property?

Let's say that a runtime class named **BookSku** has a property named **Title**. If **BookSku** raises the [INotifyPropertyChanged::PropertyChanged](#) event whenever the value of **Title** changes, then that means that **Title** is an observable property. It's the behavior of **BookSku** (raising or not raising the event) that determines which, if any, of its properties are observable.

A XAML text element, or control, can bind to, and handle, these events. Such an element or control handles the event by retrieving the updated value(s), and then updating itself to show the new value.

📌 Note

For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

Create a Blank App (Bookstore)

Begin by creating a new project in Microsoft Visual Studio. Create a **Blank App (C++/WinRT)** project, and name it *Bookstore*. Make sure that **Place solution and project**

in the same directory is unchecked. Target the latest generally-available (that is, not preview) version of the Windows SDK.

We're going to author a new class to represent a book that has an observable title property. We're authoring and consuming the class within the same compilation unit. But we want to be able to bind to this class from XAML, and for that reason it's going to be a runtime class. And we're going to use C++/WinRT to both author and consume it.

The first step in authoring a new runtime class is to add a new **Midl File (.idl)** item to the project. Name the new item `BookSku.idl`. Delete the default contents of `BookSku.idl`, and paste in this runtime class declaration.

idl

```
// BookSku.idl
namespace Bookstore
{
    runtimeclass BookSku : Windows.UI.Xaml.Data.INotifyPropertyChanged
    {
        BookSku(String title);
        String Title;
    }
}
```

ⓘ Note

Your view model classes—in fact, any runtime class that you declare in your application—need not derive from a base class. The **BookSku** class declared above is an example of that. It implements an interface, but it doesn't derive from any base class.

Any runtime class that you declare in the application that *does* derive from a base class is known as a *composable* class. And there are constraints around composable classes. For an application to pass the **Windows App Certification Kit** tests used by Visual Studio and by the Microsoft Store to validate submissions (and therefore for the application to be successfully ingested into the Microsoft Store), a composable class must ultimately derive from a Windows base class. Meaning that the class at the very root of the inheritance hierarchy must be a type originating in a `Windows.*` namespace. If you do need to derive a runtime class from a base class—for example, to implement a **BindableBase** class for all of your view models to derive from—then you can derive from **Windows.UI.Xaml.DependencyObject**.

A view model is an abstraction of a view, and so it's bound directly to the view (the XAML markup). A data model is an abstraction of data, and it's consumed only

from your view models, and not bound directly to XAML. So you can declare your data models not as runtime classes, but as C++ structs or classes. They don't need to be declared in IDL, and you're free to use whatever inheritance hierarchy you like.

Save the file, and build the project. The build won't (entirely) succeed yet, but it will do some necessary things for us. Specifically, during the build process the `midl.exe` tool is run to create a Windows Runtime metadata file that describes the runtime class (the file is placed on disk at `\Bookstore\Debug\Bookstore\Unmerged\BookSku.winmd`). Then, the `cppwinrt.exe` tool is run to generate source code files to support you in authoring and consuming your runtime class. Those files include stubs to get you started implementing the **BookSku** runtime class that you declared in your IDL. We'll find them on disk in a moment, but those stubs are `\Bookstore\Bookstore\Generated Files\sources\BookSku.h` and `BookSku.cpp`.

So now right-click the project node in Visual Studio, and click **Open Folder in File Explorer**. That opens the project folder in File Explorer. You should now be looking at the contents of the `\Bookstore\Bookstore\` folder. From there, navigate into the `\Generated Files\sources\` folder, and copy the stub files `BookSku.h` and `BookSku.cpp` to the clipboard. Navigate back up to the project folder (`\Bookstore\Bookstore\`), and paste the two files you just copied. Lastly, in **Solution Explorer** with the project node selected, make sure **Show All Files** is toggled on. Right-click the stub files that you copied, and click **Include In Project**.

Implement BookSku

Now let's open `\Bookstore\Bookstore\BookSku.h` and `BookSku.cpp` and implement our runtime class. First, you'll see a `static_assert` at the top of `BookSku.h` and `BookSku.cpp`, which you'll need to remove.

Next, in `BookSku.h`, make these changes.

- On the default constructor, change `= default` to `= delete`. That's because we don't want a default constructor.
- Add a private member to store the title string. Note that we have a constructor that takes a `winrt::hstring` value. That value is the title string.
- Add another private member for the event that we'll raise when the title changes.

After making these changes, your `BookSku.h` will look like this.

```

// BookSku.h
#pragma once
#include "BookSku.g.h"

namespace winrt::Bookstore::implementation
{
    struct BookSku : BookSkuT<BookSku>
    {
        BookSku() = delete;
        BookSku(winrt::hstring const& title);

        winrt::hstring Title();
        void Title(winrt::hstring const& value);
        winrt::event_token
        PropertyChanged(Windows::UI::Xaml::Data::PropertyChangedEventHandler const&
        value);
        void PropertyChanged(winrt::event_token const& token);

    private:
        winrt::hstring m_title;
        winrt::event<Windows::UI::Xaml::Data::PropertyChangedEventHandler>
        m_propertyChanged;
    };
}
namespace winrt::Bookstore::factory_implementation
{
    struct BookSku : BookSkuT<BookSku, implementation::BookSku>
    {
    };
}

```

In `BookSku.cpp`, implement the functions like this.

C++/WinRT

```

// BookSku.cpp
#include "pch.h"
#include "BookSku.h"
#include "BookSku.g.cpp"

namespace winrt::Bookstore::implementation
{
    BookSku::BookSku(winrt::hstring const& title) : m_title{ title }
    {
    }

    winrt::hstring BookSku::Title()
    {
        return m_title;
    }

    void BookSku::Title(winrt::hstring const& value)
    {
    }
}

```

```

    {
        if (m_title != value)
        {
            m_title = value;
            m_propertyChanged(*this,
Windows::UI::Xaml::Data::PropertyChangedEventArgs{ L"Title" });
        }
    }

    winrt::event_token
BookSku::PropertyChanged(Windows::UI::Xaml::Data::PropertyChangedEventHandle
r const& handler)
    {
        return m_propertyChanged.add(handler);
    }

    void BookSku::PropertyChanged(winrt::event_token const& token)
    {
        m_propertyChanged.remove(token);
    }
}

```

In the **Title** mutator function, we check whether a value is being set that's different from the current value. And, if so, then we update the title and also raise the [INotifyPropertyChanged::PropertyChanged](#) event with an argument equal to the name of the property that has changed. This is so that the user-interface (UI) will know which property's value to re-query.

The project will build again now, if you want to check that.

Declare and implement BookstoreViewModel

Our main XAML page is going to bind to a main view model. And that view model is going to have several properties, including one of type **BookSku**. In this step, we'll declare and implement our main view model runtime class.

Add a new **Midl File (.idl)** item named `BookstoreViewModel.idl`. But also see [Factoring runtime classes into Midl files \(.idl\)](#).

idl

```

// BookstoreViewModel.idl
import "BookSku.idl";

namespace Bookstore
{
    runtimeclass BookstoreViewModel
    {

```

```

        BookstoreViewModel();
        BookSku BookSku{ get; };
    }
}

```

Save and build (the build won't entirely succeed yet, but the reason we're building is to generate stub files again).

Copy `BookstoreViewModel.h` and `BookstoreViewModel.cpp` from the `Generated Files\sources` folder into the project folder, and include them in the project. Open those files (removing the `static_assert` again), and implement the runtime class as shown below. Note how, in `BookstoreViewModel.h`, we're including `BookSku.h`, which declares the implementation type for **BookSku** (which is `winrt::Bookstore::implementation::BookSku`). And we're removing `= default` from the default constructor.

ⓘ Note

In the listings below for `BookstoreViewModel.h` and `BookstoreViewModel.cpp`, the code illustrates the default way of constructing the `m_bookSku` data member. That's the method that dates back to the first release of C++/WinRT, and it's a good idea to be at least familiar with the pattern. With C++/WinRT version 2.0 and later, there's an optimized form of construction available to you known as *uniform construction* (see [News, and changes, in C++/WinRT 2.0](#)). Later in this topic, we'll show an example of uniform construction.

C++/WinRT

```

// BookstoreViewModel.h
#pragma once
#include "BookstoreViewModel.g.h"
#include "BookSku.h"

namespace winrt::Bookstore::implementation
{
    struct BookstoreViewModel : BookstoreViewModelT<BookstoreViewModel>
    {
        BookstoreViewModel();

        Bookstore::BookSku BookSku();

    private:
        Bookstore::BookSku m_bookSku{ nullptr };
    };
}
namespace winrt::Bookstore::factory_implementation

```

```
{
    struct BookstoreViewModel : BookstoreViewModelT<BookstoreViewModel,
implementation::BookstoreViewModel>
    {
    };
}
```

C++/WinRT

```
// BookstoreViewModel.cpp
#include "pch.h"
#include "BookstoreViewModel.h"
#include "BookstoreViewModel.g.cpp"

namespace winrt::Bookstore::implementation
{
    BookstoreViewModel::BookstoreViewModel()
    {
        m_bookSku = winrt::make<Bookstore::implementation::BookSku>
(L"Atticus");
    }

    Bookstore::BookSku BookstoreViewModel::BookSku()
    {
        return m_bookSku;
    }
}
```

❗ Note

The type of `m_bookSku` is the projected type (`winrt::Bookstore::BookSku`), and the template parameter that you use with `winrt::make` is the implementation type (`winrt::Bookstore::implementation::BookSku`). Even so, `make` returns an instance of the projected type.

The project will build again now.

Add a property of type `BookstoreViewModel` to `MainPage`

Open `MainPage.idl`, which declares the runtime class that represents our main UI page.

- Add an `import` directive to import `BookstoreViewModel.idl`.
- Add a read-only property named `MainViewModel`, of type `BookstoreViewModel`.
- Remove the `MyProperty` property.

idl

```
// MainPage.idl
import "BookstoreViewModel.idl";

namespace Bookstore
{
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();
        BookstoreViewModel MainViewModel{ get; };
    }
}
```

Save the file. The project won't entirely succeed in building yet, but building now is a useful thing to do because it regenerates the source code files in which the **MainPage** runtime class is implemented (`\Bookstore\Bookstore\Generated Files\sources\MainPage.h` and `MainPage.cpp`). So go ahead and build now. The build error you can expect to see at this stage is **'MainViewModel': is not a member of 'winrt::Bookstore::implementation::MainPage'**.

If you omit the include of `BookstoreViewModel.idl` (see the listing of `MainPage.idl` above), then you'll see the error **expecting < near "MainViewModel"**. Another tip is to make sure that you leave all types in the same namespace—the namespace that's shown in the code listings.

To resolve the error that we expect to see, you'll now need to copy the accessor stubs for the **MainViewModel** property out of the generated files (`\Bookstore\Bookstore\Generated Files\sources\MainPage.h` and `MainPage.cpp`) and into `\Bookstore\Bookstore\MainPage.h` and `MainPage.cpp`. The steps to do that are described next.

In `\Bookstore\Bookstore\MainPage.h`, perform these steps.

- Include `BookstoreViewModel.h`, which declares the implementation type for **BookstoreViewModel** (which is **winrt::Bookstore::implementation::BookstoreViewModel**).
- Add a private member to store the view model. Note that the property accessor function (and the member `m_mainViewModel`) is implemented in terms of the projected type for **BookstoreViewModel** (which is **Bookstore::BookstoreViewModel**).
- The implementation type is in the same project (compilation unit) as the application, so we construct `m_mainViewModel` via the constructor overload that takes `std::nullptr_t`.

- Remove the **MyProperty** property.

❗ Note

In the pair of listings below for `MainPage.h` and `MainPage.cpp`, the code illustrates the default way of constructing the `m_mainViewModel` data member. In the section that follows, we'll show a version that uses uniform construction instead.

C++/WinRT

```
// MainPage.h
...
#include "BookstoreViewModel.h"
...
namespace winrt::Bookstore::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        MainPage();

        Bookstore::BookstoreViewModel MainViewModel();

        void ClickHandler(Windows::Foundation::IIInspectable const&,
            Windows::UI::Xaml::RoutedEventArgs const&);

    private:
        Bookstore::BookstoreViewModel m_mainViewModel{ nullptr };
    };
}
```

In `\Bookstore\Bookstore\MainPage.cpp`, as shown in the listing below, make the following changes.

- Call `winrt::make` (with the `BookstoreViewModel` implementation type) to assign a new instance of the projected `BookstoreViewModel` type to `m_mainViewModel`. As we saw above, the `BookstoreViewModel` constructor creates a new `BookSku` object as a private data member, setting its title initially to `L"Atticus"`.
- In the button's event handler (`ClickHandler`), update the book's title to its published title.
- Implement the accessor for the `MainViewModel` property.
- Remove the **MyProperty** property.

C++/WinRT

```

// MainPage.cpp
#include "pch.h"
#include "MainPage.h"
#include "MainPage.g.cpp"

using namespace winrt;
using namespace Windows::UI::Xaml;

namespace winrt::Bookstore::implementation
{
    MainPage::MainPage()
    {
        m_mainViewModel =
winrt::make<Bookstore::implementation::BookstoreViewModel>();
        InitializeComponent();
    }

    void MainPage::ClickHandler(Windows::Foundation::IIInspectable const& /*
sender */, Windows::UI::Xaml::RoutedEventArgs const& /* args */)
    {
        MainViewModel().BookSku().Title(L"To Kill a Mockingbird");
    }

    Bookstore::BookstoreViewModel MainPage::MainViewModel()
    {
        return m_mainViewModel;
    }
}

```

Uniform construction

To use uniform construction instead of `winrt::make`, in `MainPage.h` declare and initialize `m_mainViewModel` in just one step, as shown below.

C++/WinRT

```

// MainPage.h
...
#include "BookstoreViewModel.h"
...
struct MainPage : MainPageT<MainPage>
{
    ...
private:
    Bookstore::BookstoreViewModel m_mainViewModel;
};
...

```

And then, in the **MainPage** constructor in `MainPage.cpp`, there's no need for the code

```
m_mainViewModel = winrt::make<Bookstore::implementation::BookstoreViewModel>();
```

For more info about uniform construction, and code examples, see [Opt in to uniform construction, and direct implementation access](#).

Bind the button to the Title property

Open `MainPage.xaml`, which contains the XAML markup for our main UI page. As shown in the listing below, remove the name from the button, and change its **Content** property value from a literal to a binding expression. Note the `Mode=OneWay` property on the binding expression (one-way from the view model to the UI). Without that property, the UI will not respond to property changed events.

XAML

```
<Button Click="ClickHandler" Content="{x:Bind MainViewModel.BookSku.Title,
Mode=OneWay}" />
```

Now build and run the project. Click the button to execute the **Click** event handler. That handler calls the book's title mutator function; that mutator raises an event to let the UI know that the **Title** property has changed; and the button re-queries that property's value to update its own **Content** value.

Using the {Binding} markup extension with C++/WinRT

For the currently released version of C++/WinRT, in order to be able to use the {Binding} markup extension you'll need to implement the [ICustomPropertyProvider](#) and [ICustomProperty](#) interfaces.

Element-to-element binding

You can bind the property of one XAML element to the property of another XAML element. Here's an example of how that looks in markup.

XAML

```
<TextBox x:Name="myTextBox" />
<TextBlock Text="{x:Bind myTextBox.Text, Mode=OneWay}" />
```

You'll need to declare the named XAML entity `myTextBox` as a read-only property in your Midl file (.idl).

idl

```
// MainPage.idl
runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
{
    MainPage();
    Windows.UI.Xaml.Controls.TextBox myTextBox{ get; };
}
```

Here's the reason for this necessity. All types that the XAML compiler needs to validate (including those used in `{x:Bind}`) are read from Windows Metadata (WinMD). All you need to do is to add the read-only property to your Midl file. Don't implement it, because the autogenerated XAML code-behind provides the implementation for you.

Consuming objects from XAML markup

All entities consumed by using the XAML `{x:Bind}` [markup extension](#) must be exposed publicly in IDL. Furthermore, if XAML markup contains a reference to another element that's also in markup, then the getter for that markup must be present in IDL.

XAML

```
<Page x:Name="MyPage">
    <StackPanel>
        <CheckBox x:Name="UseCustomColorCheckBox" Content="Use custom color"
            Click="UseCustomColorCheckBox_Click" />
        <Button x:Name="ChangeColorButton" Content="Change color"
            Click="{x:Bind ChangeColorButton_OnClick}"
            IsEnabled="{x:Bind UseCustomColorCheckBox.IsChecked.Value,
Mode=OneWay}" />
    </StackPanel>
</Page>
```

The *ChangeColorButton* element refers to the *UseCustomColorCheckBox* element via binding. So the IDL for this page must declare a read-only property named *UseCustomColorCheckBox* in order for it to be accessible to binding.

The click event handler delegate for *UseCustomColorCheckBox* uses classic XAML delegate syntax, so that doesn't need an entry in the IDL; it just needs to be public in your implementation class. On the other hand, *ChangeColorButton* also has an `{x:Bind}` click event handler, which must also go into the IDL.

idl

```
runtimeclass MyPage : Windows.UI.Xaml.Controls.Page
{
    MyPage();

    // These members are consumed by binding.
    void ChangeColorButton_OnClick();
    Windows.UI.Xaml.Controls.CheckBox UseCustomColorCheckBox{ get; };
}
```

You don't need to provide an implementation for the **UseCustomColorCheckBox** property. The XAML code generator does that for you.

Binding to Boolean

You might do this in a diagnostic mode:

XAML

```
<TextBlock Text="{Binding CanPair}"/>
```

That displays `true` or `false` in C++/CX; but it displays `Windows.Foundation.IReference`1<Boolean>` in C++/WinRT.

Instead, use `x:Bind` when binding to a Boolean.

XAML

```
<TextBlock Text="{x:Bind CanPair}"/>
```

Using the Windows Implementation Libraries (WIL)

The [Windows Implementation Libraries \(WIL\)](#) provides helpers to ease writing bindable properties. See [Notifying Properties](#) in the WIL documentation.

Important APIs

- [INotifyPropertyChanged::PropertyChanged](#)
- [winrt::make function template](#)

Related topics

- [Consume APIs with C++/WinRT](#)
- [Author APIs with C++/WinRT](#)

XAML items controls; bind to a C++/WinRT collection

Article • 10/27/2022

A collection that can be effectively bound to a XAML items control is known as an *observable* collection. This idea is based on the software design pattern known as the *observer pattern*. This topic shows how to implement observable collections in [C++/WinRT](#), and how to bind XAML items controls to them (for background info, see [Data binding](#)).

If you want to follow along with this topic, then we recommend that you first create the project that's described in [XAML controls; bind to a C++/WinRT property](#). This topic adds more code to that project, and it adds to the concepts explained in that topic.

📘 Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with C++/WinRT, see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

What does *observable* mean for a collection?

If a runtime class that represents a collection chooses to raise the [IObservableVector<T>::VectorChanged](#) event whenever an element is added to it or removed from it, then the runtime class is an observable collection. A XAML items control can bind to, and handle, these events by retrieving the updated collection and then updating itself to show the current elements.

📝 Note

For info about installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

Add a BookSkus collection to BookstoreViewModel

In [XAML controls; bind to a C++/WinRT property](#), we added a property of type **BookSku** to our main view model. In this step, we'll use the [winrt::single_threaded_observable_vector](#) factory function template to help us implement an observable collection of **BookSku** on the same view model.

Declare a new property in `BookstoreViewModel.idl`.

```
idl

// BookstoreViewModel.idl
...
runtimeclass BookstoreViewModel
{
    BookSku BookSku{ get; };
    Windows.Foundation.Collections.IObservableVector<BookSku> BookSkus{ get; };
};
}
...
```

ⓘ Note

In the MIDL 3.0 listing above, note that the type of the **BookSkus** property is **IObservableVector** of **BookSku**. In the next section of this topic, we'll be binding the items source of a **ListBox** to **BookSkus**. A list box is an items control, and to correctly set the **ItemsControl.ItemsSource** property, you need to set it to a value of type **IObservableVector**, or **IVector**, or of an interoperability type such as **IBindableObservableVector**.

⚠ Warning

The code shown in this topic applies to C++/WinRT version 2.0.190530.8 or later. If you're using an earlier version, then you'll need to make some minor tweaks to the code shown. In the MIDL 3.0 listing above, change the **BookSkus** property to **IObservableVector** of **IInspectable**. And then use **IInspectable** (instead of **BookSku**) in your implementation, too.

Save and build. Copy the accessor stubs from `BookstoreViewModel.h` and `BookstoreViewModel.cpp` in the `\Bookstore\Bookstore\Generated Files\sources` folder (for more details, see the previous topic, [XAML controls; bind to a C++/WinRT property](#)). Implement those accessor stubs like this.

```
// BookstoreViewModel.h
...
struct BookstoreViewModel : BookstoreViewModelT<BookstoreViewModel>
{
    BookstoreViewModel();

    Bookstore::BookSku BookSku();

    Windows::Foundation::Collections::IObservableVector<Bookstore::BookSku>
    BookSkus();

private:
    Bookstore::BookSku m_bookSku{ nullptr };
    Windows::Foundation::Collections::IObservableVector<Bookstore::BookSku>
    m_bookSkus;
};
...
```

C++/WinRT

```
// BookstoreViewModel.cpp
...
BookstoreViewModel::BookstoreViewModel()
{
    m_bookSku = winrt::make<Bookstore::implementation::BookSku>(L"Atticus");
    m_bookSkus =
    winrt::single_threaded_observable_vector<Bookstore::BookSku>();
    m_bookSkus.Append(m_bookSku);
}

Bookstore::BookSku BookstoreViewModel::BookSku()
{
    return m_bookSku;
}

Windows::Foundation::Collections::IObservableVector<Bookstore::BookSku>
BookstoreViewModel::BookSkus()
{
    return m_bookSkus;
}
...
```

Bind a ListBox to the BookSkus property

Open `MainPage.xaml`, which contains the XAML markup for our main UI page. Add the following markup inside the same `StackPanel` as the `Button`.

XAML

```

<ListBox ItemsSource="{x:Bind MainViewModel.BookSkus}">
    <ItemsControl.ItemTemplate>
        <DataTemplate x:DataType="local:BookSku">
            <TextBlock Text="{x:Bind Title, Mode=OneWay}" />
        </DataTemplate>
    </ItemsControl.ItemTemplate>
</ListBox>

```

In `MainPage.cpp`, add a line of code to the **Click** event handler to append a book to the collection.

C++/WinRT

```

// MainPage.cpp
...
void MainPage::ClickHandler(IInspectable const&, RoutedEventArgs const&)
{
    MainViewModel().BookSku().Title(L"To Kill a Mockingbird");

    MainViewModel().BookSkus().Append(winrt::make<Bookstore::implementation::BookSku>(L"Moby Dick"));
}
...

```

Now build and run the project. Click the button to execute the **Click** event handler. We saw that the implementation of **Append** raises an event to let the UI know that the collection has changed; and the **ListBox** re-queries the collection to update its own **Items** value. Just as before, the title of one of the books changes; and that title change is reflected both on the button and in the list box.

Important APIs

- [IObservableVector<T>::VectorChanged](#)
- [winrt::make function template](#)

Related topics

- [Consume APIs with C++/WinRT](#)
- [Author APIs with C++/WinRT](#)

XAML custom (templated) controls with C++/WinRT

Article • 10/20/2022

Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with C++/WinRT, see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

One of the most powerful features of the Universal Windows Platform (UWP) is the flexibility that the user-interface (UI) stack provides to create custom controls based on the XAML [Control](#) type. The XAML UI framework provides features such as [custom dependency properties](#) and [attached properties](#), and [control templates](#), which make it easy to create feature-rich and customizable controls. This topic walks you through the steps of creating a custom (templated) control with C++/WinRT.

Create a Blank App (BgLabelControlApp)

Begin by creating a new project in Microsoft Visual Studio. Create a **Blank App (C++/WinRT)** project, set its name to *BgLabelControlApp*, and (so that your folder structure will match the walkthrough) make sure that **Place solution and project in the same directory** is unchecked. Target the latest generally-available (that is, not preview) version of the Windows SDK.

In a later section of this topic, you'll be directed to build your project (but don't build until then).

Note

For info about setting up Visual Studio for C++/WinRT development—including installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support)—see [Visual Studio support for C++/WinRT](#).

We're going to author a new class to represent a custom (templated) control. We're authoring and consuming the class within the same compilation unit. But we want to be

able to instantiate this class from XAML markup, and for that reason it's going to be a runtime class. And we're going to use C++/WinRT to both author and consume it.

The first step in authoring a new runtime class is to add a new **Midl File (.idl)** item to the project. Name it `BgLabelControl.idl`. Delete the default contents of `BgLabelControl.idl`, and paste in this runtime class declaration.

```
idl

// BgLabelControl.idl
namespace BgLabelControlApp
{
    runtimeclass BgLabelControl : Windows.UI.Xaml.Controls.Control
    {
        BgLabelControl();
        static Windows.UI.Xaml.DependencyProperty LabelProperty{ get; };
        String Label;
    }
}
```

The listing above shows the pattern that you follow when declaring a dependency property (DP). There are two pieces to each DP. First, you declare a read-only static property of type [DependencyProperty](#). It has the name of your DP plus *Property*. You'll use this static property in your implementation. Second, you declare a read-write instance property with the type and name of your DP. If you wish to author an *attached property* (rather than a DP), then see the code examples in [Custom attached properties](#).

ⓘ Note

If you want a DP with a floating-point type, then make it `double` (`Double` in **MIDL 3.0**). Declaring and implementing a DP of type `float` (`Single` in **MIDL**), and then setting a value for that DP in XAML markup, results in the error *Failed to create a 'Windows.Foundation.Single' from the text '<NUMBER>'*.

Save the file. The project won't build to completion at the moment, but building now is a useful thing to do because it generates the source code files in which you'll implement the **BgLabelControl** runtime class. So go ahead and build now (the build errors you can expect to see at this stage have to do with an "unresolved external symbol").

During the build process, the `midl.exe` tool is run to create a Windows Runtime metadata file

(`\BgLabelControlApp\Debug\BgLabelControlApp\Unmerged\BgLabelControl.winmd`)

describing the runtime class. Then, the `cppwinrt.exe` tool is run to generate source code

files to support you in authoring and consuming your runtime class. These files include stubs to get you started implementing the **BgLabelControl** runtime class that you declared in your IDL. Those stubs are `\BgLabelControlApp\BgLabelControlApp\Generated Files\sources\BgLabelControl.h` and `BgLabelControl.cpp`.

Copy the stub files `BgLabelControl.h` and `BgLabelControl.cpp` from `\BgLabelControlApp\BgLabelControlApp\Generated Files\sources\` into the project folder, which is `\BgLabelControlApp\BgLabelControlApp\`. In **Solution Explorer**, make sure **Show All Files** is toggled on. Right-click the stub files that you copied, and click **Include In Project**.

You'll see a `static_assert` at the top of `BgLabelControl.h` and `BgLabelControl.cpp`, which you'll need to remove. Now the project will build.

Implement the BgLabelControl custom control class

Now, let's open `\BgLabelControlApp\BgLabelControlApp\BgLabelControl.h` and `BgLabelControl.cpp` and implement our runtime class. In `BgLabelControl.h`, change the constructor to set the default style key, implement **Label** and **LabelProperty**, add a static event handler named **OnLabelChanged** to process changes to the value of the dependency property, and add a private member to store the backing field for **LabelProperty**.

After adding those, your `BgLabelControl.h` looks like this. You can copy and paste this code listing to replace the contents of `BgLabelControl.h`.

C++/WinRT

```
// BgLabelControl.h
#pragma once
#include "BgLabelControl.g.h"

namespace winrt::BgLabelControlApp::implementation
{
    struct BgLabelControl : BgLabelControlT<BgLabelControl>
    {
        BgLabelControl() {
            DefaultStyleKey(winrt::box_value(L"BgLabelControlApp.BgLabelControl")); }

        winrt::hstring Label()
        {
            return winrt::unbox_value<winrt::hstring>
                (GetValue(m_labelProperty));
        }
    }
}
```

```

        void Label(winrt::hstring const& value)
        {
            SetValue(m_labelProperty, winrt::box_value(value));
        }

        static Windows::UI::Xaml::DependencyProperty LabelProperty() {
return m_labelProperty; }

        static void OnLabelChanged(Windows::UI::Xaml::DependencyObject
const&, Windows::UI::Xaml::DependencyPropertyChangedEventArgs const&);

    private:
        static Windows::UI::Xaml::DependencyProperty m_labelProperty;
    };
}
namespace winrt::BgLabelControlApp::factory_implementation
{
    struct BgLabelControl : BgLabelControlT<BgLabelControl,
implementation::BgLabelControl>
    {
    };
}

```

In `BgLabelControl.cpp`, define the static members like this. You can copy and paste this code listing to replace the contents of `BgLabelControl.cpp`.

C++/WinRT

```

// BgLabelControl.cpp
#include "pch.h"
#include "BgLabelControl.h"
#include "BgLabelControl.g.cpp"

namespace winrt::BgLabelControlApp::implementation
{
    Windows::UI::Xaml::DependencyProperty BgLabelControl::m_labelProperty =
        Windows::UI::Xaml::DependencyProperty::Register(
            L"Label",
            winrt::xaml_typename<winrt::hstring>(),
            winrt::xaml_typename<BgLabelControlApp::BgLabelControl>(),
            Windows::UI::Xaml::PropertyMetadata{ winrt::box_value(L"default
label"), Windows::UI::Xaml::PropertyChangedCallback{
&BgLabelControl::OnLabelChanged } }
        );

    void BgLabelControl::OnLabelChanged(Windows::UI::Xaml::DependencyObject
const& d, Windows::UI::Xaml::DependencyPropertyChangedEventArgs const& /* e
*/)
    {
        if (BgLabelControlApp::BgLabelControl theControl{
d.try_as<BgLabelControlApp::BgLabelControl>() })
        {

```

```

        // Call members of the projected type via theControl.

        BgLabelControlApp::implementation::BgLabelControl* ptr{
winrt::get_self<BgLabelControlApp::implementation::BgLabelControl>
(theControl) };
        // Call members of the implementation type via ptr.
    }
}
}

```

In this walkthrough, we won't be using **OnLabelChanged**. But it's there so that you can see how to register a dependency property with a property-changed callback. The implementation of **OnLabelChanged** also shows how to obtain a derived projected type from a base projected type (the base projected type is **DependencyObject**, in this case). And it shows how to then obtain a pointer to the type that implements the projected type. That second operation will naturally only be possible in the project that implements the projected type (that is, the project that implements the runtime class).

ⓘ Note

If you haven't installed the Windows SDK version 10.0.17763.0 (Windows 10, version 1809), or later, then you need to call `winrt::from_abi` in the dependency property changed event handler above, instead of `winrt::get_self`.

Design the default style for BgLabelControl

In its constructor, **BgLabelControl** sets a default style key for itself. But what is a default style? A custom (templated) control needs to have a default style—containing a default control template—which it can use to render itself with in case the consumer of the control doesn't set a style and/or template. In this section we'll add a markup file to the project containing our default style.

Make sure that **Show All Files** is still toggled on (in **Solution Explorer**). Under your project node, create a new folder (not a filter, but a folder) and name it "Themes". Under **Themes**, add a new item of type **Visual C++ > XAML > XAML View**, and name it "Generic.xaml". The folder and file names have to be like this in order for the XAML framework to find the default style for a custom control. Delete the default contents of **Generic.xaml**, and paste in the markup below.

XAML

```

<!-- \Themes\Generic.xaml -->
<ResourceDictionary

```

```

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:BgLabelControlApp">

<Style TargetType="local:BgLabelControl" >
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="local:BgLabelControl">
                <Grid Width="100" Height="100" Background="
{TemplateBinding Background}">
                    <TextBlock HorizontalAlignment="Center"
VerticalAlignment="Center" Text="{TemplateBinding Label}"/>
                </Grid>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>
</ResourceDictionary>

```

In this case, the only property that the default style sets is the control template. The template consists of a square (whose background is bound to the **Background** property that all instances of the XAML **Control** type have), and a text element (whose text is bound to the **BgLabelControl::Label** dependency property).

Add an instance of BgLabelControl to the main UI page

Open `MainPage.xaml`, which contains the XAML markup for our main UI page. Immediately after the **Button** element (inside the **StackPanel**), add the following markup.

XAML

```
<local:BgLabelControl Background="Red" Label="Hello, World!"/>
```

Also, add the following include directive to `MainPage.h` so that the **MainPage** type (a combination of compiling XAML markup and imperative code) is aware of the **BgLabelControl** custom control type. If you want to use **BgLabelControl** from another XAML page, then add this same include directive to the header file for that page, too. Or, alternatively, just put a single include directive in your precompiled header file.

C++/WinRT

```
// MainPage.h
...
```

```
#include "BgLabelControl.h"
...
```

Now build and run the project. You'll see that the default control template is binding to the background brush, and to the label, of the **BgLabelControl** instance in the markup.

This walkthrough showed a simple example of a custom (templated) control in C++/WinRT. You can make your own custom controls arbitrarily rich and full-featured. For example, a custom control can take the form of something as complicated as an editable data grid, a video player, or a visualizer of 3D geometry.

Implementing *overridable* methods, such as **MeasureOverride** and **OnApplyTemplate**

See the section in [Calling and overriding your base type with C++/WinRT](#).

Important APIs

- [Control class](#)
- [DependencyProperty class](#)
- [FrameworkElement class](#)
- [UIElement class](#)

Related topics

- [Control templates](#)
- [Custom dependency properties](#)

Calling and overriding your base type with C++/WinRT

Article • 09/11/2023

Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with [C++/WinRT](#), see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

Implementing *overridable* methods, such as `MeasureOverride` and `OnApplyTemplate`

There are some extension points in XAML that your application can plug into, for example:

- [MeasureOverride](#)
- [OnApplyTemplate](#)
- [GoToStateCore](#)

You derive a custom control from the [Control](#) runtime class, which itself further derives from base runtime classes. And there are `overridable` methods of [Control](#), [FrameworkElement](#), and [UIElement](#) that you can override in your derived class. Here's a code example showing you how to do that.

C++/WinRT

```
struct BgLabelControl : BgLabelControlT<BgLabelControl>
{
    ...
    // Control overrides.
    void OnPointerPressed(Windows::UI::Xaml::Input::PointerRoutedEventArgs
const& /* e */) const { ... };

    // FrameworkElement overrides.
    Windows::Foundation::Size MeasureOverride(Windows::Foundation::Size
const& /* availableSize */) const { ... };
    void OnApplyTemplate() const { ... };

    // UIElement overrides.
    Windows::UI::Xaml::Automation::Peers::AutomationPeer
OnCreateAutomationPeer() const { ... };
```

```
...  
};
```

Overridable methods present themselves differently in different language projections. In C#, for example, overridable methods typically appear as protected virtual methods. In C++/WinRT, they're neither virtual nor protected, but you can still override them and provide your own implementation, as shown above.

If you're overriding one of these overridable methods in C++/WinRT, then your `runtimeclass` IDL mustn't declare the method. For more info about the `base_type` syntax shown, see the next section in this topic ([Calling your base type](#)).

IDL

```
idl
```

```
namespace Example  
{  
    runtimeclass CustomVSM : Windows.UI.Xaml.VisualStateManager  
    {  
        CustomVSM();  
        // note that we don't declare GoToStateCore here  
    }  
}
```

C++/WinRT

```
C++/WinRT
```

```
namespace winrt::Example::implementation  
{  
    struct CustomVSM : CustomVSMT<CustomVSM>  
    {  
        CustomVSM() {}  
  
        bool GoToStateCore(winrt::Windows::UI::Xaml::Controls::Control  
const& control, winrt::Windows::UI::Xaml::FrameworkElement const&  
templateRoot, winrt::hstring const& stateName,  
winrt::Windows::UI::Xaml::VisualStateGroup const& group,  
winrt::Windows::UI::Xaml::VisualState const& state, bool useTransitions) {  
            return base_type::GoToStateCore(control, templateRoot,  
stateName, group, state, useTransitions);  
        }  
    };  
}
```

Calling your base type

You can access your base type, and call methods on it, by using the type alias `base_type`. We saw an example of this in the previous section; but you can use `base_type` to access any base class member (not just overridden methods). Here's an example:

C++/WinRT

```
struct MyDerivedRuntimeClass : MyDerivedRuntimeClassT<MyDerivedRuntimeClass>
{
    ...

    void Foo()
    {
        // Call my base type's Bar method.
        base_type::Bar();
    }
};
```

Important APIs

- [Control class](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Passing parameters to projected APIs

Article • 11/15/2023

For certain types, C++/WinRT provides alternative methods for passing a parameter to a projected API. These parameter-accepting classes are placed in the `winrt::param` namespace. Only C++/WinRT-generated code should use these classes; don't use them in your own functions and methods.

Important

You shouldn't use the types in the `winrt::param` namespace yourself. They're for the benefit of the projection.

Some of these alternative distinguish between synchronous and asynchronous calls. The version for asynchronous calls typically takes ownership of the parameter data to ensure that the values remain valid and unchanged until the asynchronous call completes. Note, however, that this protection does not extend to changes to the collection from another thread. Preventing that is your responsibility.

Alternatives for string parameters

`winrt::param::hstring` simplifies passing parameters as `winrt::hstring`. In addition to `winrt::hstring`, these alternatives are also accepted:

 Expand table

Alternative	Notes
<code>{}</code>	An empty string.
<code>std::wstring_view</code>	The view must be followed by a null terminator.
<code>std::wstring</code>	
<code>wchar_t const*</code>	A null-terminated string.

You can't pass `nullptr` to represent the empty string. Instead, use `L""` or `{}`.

The compiler knows how to evaluate `wcslen` on string literals at compile time. So, for literals, `L"Name"sv` and `L"Name"` are equivalent.

Note that `std::wstring_view` objects are not null-terminated, but C++/WinRT requires that the character after the end of the view be a null. If you pass a non-null-terminated `std::wstring_view`, then the process will terminate.

Alternatives for iterable parameters

`winrt::param::iterable<T>` and `winrt::param::async_iterable<T>` simplify passing parameters as `Iterable<T>`.

The Windows Runtime collections `IVector<T>` and `IVectorView<T>` already support `Iterable<T>`. The Windows Runtime collections `IMap<K, V>` and `IMapView<K, V>` already support `Iterable<KeyValuePair<K, V>>`.

In addition to `Iterable<T>`, the following alternatives are also accepted. Note that some alternatives are available only for synchronous methods.

[Expand table](#)

Alternative	Sync	Async	Notes
<code>std::vector<T> const&</code>	Yes	No	
<code>std::vector<T>&&</code>	Yes	Yes	Contents are moved into a temporary iterable.
<code>std::initializer_list<T></code>	Yes	Yes	Async version copies the items.
<code>std::initializer_list<U></code>	Yes	No	<code>U</code> must be convertible to <code>T</code> .
<code>{ begin, end }</code>	Yes	No	<code>begin</code> and <code>end</code> must be forward iterators , and <code>*begin</code> must be convertible to <code>T</code> .

The double-iterator works more generally for the case where you have a collection that doesn't fit any of the scenarios above, as long as you can iterate over it and produce things that can be converted to `T`. For example, you may have a `IVector<U>` or `std::vector<U>`, where `U` is convertible to `T`.

In the following example, the `SetStorageItems` method expects an `Iterable<IStorageItem>`. The double-iterator pattern lets us pass other types of collections.

C++/WinRT

```
// IVector of derived types.  
winrt::Windows::Foundation::Collections::IVector<winrt::Windows::Storage::StorageFile>
```

```

storageFiles{ /* initialization elided */ };
dataPackage.SetStorageItems(storageFiles); // doesn't work
dataPackage.SetStorageItems({ storageFiles.begin(), storageFiles.end() });
// works

// Array of derived types.
std::array<winrt::Windows::Storage::StorageFile, 3>
    storageFiles{ /* initialization elided */ };
dataPackage.SetStorageItems(storageFiles); // doesn't work
dataPackage.SetStorageItems({ storageFiles.begin(), storageFiles.end() });
// works

```

For the case of `Iterable<KeyValuePair<K, V>>`, the following alternatives are accepted. Note that some alternatives are available only for synchronous methods.

[Expand table](#)

Alternative	Sync	Async	Notes
<code>std::map<K, V> const&</code>	Yes	No	
<code>std::map<K, V>&&</code>	Yes	Yes	Contents are moved into a temporary iterable.
<code>std::unordered_map<K, V> const&</code>	Yes	No	
<code>std::unordered_map<K, V>&&</code>	Yes	Yes	Contents are moved into a temporary iterable.
<code>std::initializer_list<std::pair<K, V>></code>	Yes	Yes	Async version copies the list into a temporary iterable.
<code>{ begin, end }</code>	Yes	No	<code>begin</code> and <code>end</code> must be forward iterators [↗] , and <code>begin->first</code> and <code>begin->second</code> must be convertible to <code>K</code> and <code>V</code> , respectively.

Alternatives for vector view parameters

`winrt::param::vector_view<T>` and `winrt::param::async_vector_view<T>` simplify passing parameters as `IVectorView<T>`.

You can call `IVector<T>::GetView` to get an `IVectorView<T>` from an `IVector<T>`.

In addition to `IVectorView<T>`, the following alternatives are also accepted. Note that some alternatives are available only for synchronous methods.

[Expand table](#)

Alternative	Sync	Async	Notes
<code>std::vector<T> const&</code>	Yes	No	
<code>std::vector<T>&&</code>	Yes	Yes	Contents are moved into a temporary view.
<code>std::initializer_list<T></code>	Yes	Yes	Async version copies the list into a temporary view.
<code>{ begin, end }</code>	Yes	No	<code>begin</code> and <code>end</code> must be forward iterators , and <code>*begin</code> must be convertible to <code>T</code> .

Again, the double-iterator version can be used to create vector views out of things that don't fit an existing alternative. The temporary view is more efficient if the `begin` and `end` iterators are random-access iterators.

Alternatives for map view parameters

`wintr::param::map_view<T>` and `wintr::param::async_map_view<T>` simplify passing parameters as `IMapView<T>`.

You can call `IMap<K, V>::GetView` to get an `IMapView<K, V>` from an `IMap<K, V>`.

In addition to `IMapView<K, V>`, the following alternatives are also accepted. Note that some alternatives are available only for synchronous methods.

[Expand table](#)

Alternative	Sync	Async	Notes
<code>std::map<K, V> const&</code>	Yes	No	
<code>std::map<K, V>&&</code>	Yes	Yes	Contents are moved into a temporary view.
<code>std::unordered_map<K, V> const&</code>	Yes	No	
<code>std::unordered_map<K, V>&&</code>	Yes	Yes	Contents are moved into a temporary view.
<code>std::initializer_list<std::pair<K, V>></code>	Yes	Yes	Contents are copied into a temporary view. Keys may not be duplicated.

Alternatives for vector parameters

`wintr::param::vector<T>` simplifies passing parameters as `IVector<T>`. In addition to `IVector<T>`, these alternatives are also accepted:

[Expand table](#)

Alternative	Notes
<code>std::vector<T>&&</code>	Contents are moved into a temporary vector. Results are <i>not</i> moved back.
<code>std::initializer_list<T></code>	

If the method mutates the temporary vector, then those changes are not reflected in the original parameters. To observe the changes, pass an `IVector<T>`.

Alternatives for map parameters

`wintr::param::map<K, V>` simplifies passing parameters as `IMap<K, V>`. In addition to `IMap<K, V>`, these alternatives are also accepted:

[Expand table](#)

You can pass	Notes
<code>std::map<K, V>&&</code>	Contents are moved into a temporary map. Results are <i>not</i> moved back.
<code>std::unordered_map<K, V>&&</code>	Contents are moved into a temporary map. Results are <i>not</i> moved back.
<code>std::initializer_list<std::pair<K, V>></code>	

If the method mutates the temporary map, then those changes are not reflected in the original parameters. To observe the changes, pass an `IMap<K, V>`.

Alternatives for array parameters

`wintr::array_view<T>` is not in the `wintr::param` namespace, but it is used for parameters that are C-style arrays. In addition to an explicit `array_view<T>`, these alternatives are also accepted:

[Expand table](#)

Alternative	Notes
<code>{}</code>	Empty array.
<code>U[]</code>	A C-style array, where <code>U</code> is convertible to <code>T</code> , and <code>sizeof(U) == sizeof(T)</code> .

Alternative	Notes
<code>std::array<U, N></code>	Where U is convertible to T , and <code>sizeof(U) == sizeof(T)</code> .
<code>std::vector<U></code>	Where U is convertible to T , and <code>sizeof(U) == sizeof(T)</code> .
<code>{ begin, end }</code>	<code>begin</code> and <code>end</code> must be of type T* , representing the range <code>[begin, end)</code> .
<code>std::initializer_list<T></code>	
<code>std::span<U, N></code>	Where U is convertible to T , and <code>sizeof(U) == sizeof(T)</code> .

Also see the blog post [The various patterns for passing C-style arrays across the Windows Runtime ABI boundary](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Consume COM components with C++/WinRT

Article • 10/20/2022

You can use the facilities of the [C++/WinRT](#) library to consume COM components, such as the high-performance 2-D and 3-D graphics of the DirectX APIs. C++/WinRT is the simplest way to use DirectX without compromising performance. This topic uses a Direct2D code example to show how to use C++/WinRT to consume COM classes and interfaces. You can, of course, mix COM and Windows Runtime programming within the same C++/WinRT project.

At the end of this topic, you'll find a full source code listing of a minimal Direct2D application. We'll lift excerpts from that code and use them to illustrate how to consume COM components using C++/WinRT using various facilities of the C++/WinRT library.

COM smart pointers ([winrt::com_ptr](#))

When you program with COM, you work directly with interfaces rather than with objects (that's also true behind the scenes for Windows Runtime APIs, which are an evolution of COM). To call a function on a COM class, for example, you activate the class, get an interface back, and then you call functions on that interface. To access the state of an object, you don't access its data members directly; instead, you call accessor and mutator functions on an interface.

To be more specific, we're talking about interacting with interface *pointers*. And for that, we benefit from the existence of the COM smart pointer type in C++/WinRT—the [winrt::com_ptr](#) type.

C++/WinRT

```
#include <d2d1_1.h>
...
winrt::com_ptr<ID2D1Factory1> factory;
```

The code above shows how to declare an uninitialized smart pointer to a [ID2D1Factory1](#) COM interface. The smart pointer is uninitialized, so it's not yet pointing to a [ID2D1Factory1](#) interface belonging to any actual object (it's not pointing to an interface at all). But it has the potential to do so; and (being a smart pointer) it has the ability via COM reference counting to manage the lifetime of the owning object of the interface that it points to, and to be the medium by which you call functions on that interface.

COM functions that return an interface pointer as void

You can call the `com_ptr::put_void` function to write to an uninitialized smart pointer's underlying raw pointer.

C++/WinRT

```
D2D1_FACTORY_OPTIONS options{ D2D1_DEBUG_LEVEL_NONE };
D2D1CreateFactory(
    D2D1_FACTORY_TYPE_SINGLE_THREADED,
    __uuidof(factory),
    &options,
    factory.put_void()
);
```

The code above calls the `D2D1CreateFactory` function, which returns an `ID2D1Factory1` interface pointer via its last parameter, which has `void**` type. Many COM functions return a `void**`. For such functions, use `com_ptr::put_void` as shown.

COM functions that return a specific interface pointer

The `D3D11CreateDevice` function returns an `ID3D11Device` interface pointer via its third-from-last parameter, which has `ID3D11Device**` type. For functions that return a specific interface pointer like that, use `com_ptr::put`.

C++/WinRT

```
wintrt::com_ptr<ID3D11Device> device;
D3D11CreateDevice(
    ...
    device.put(),
    ...);
```

The code example in the section before this one shows how to call the raw `D2D1CreateFactory` function. But in fact, when the code example for this topic calls `D2D1CreateFactory`, it uses a helper function template that wraps the raw API, and so the code example actually uses `com_ptr::put`.

C++/WinRT

```
wintrt::com_ptr<ID2D1Factory1> factory;
D2D1CreateFactory(
```

```
D2D1_FACTORY_TYPE_SINGLE_THREADED,  
options,  
factory.put());
```

COM functions that return an interface pointer as IUnknown

The [DWriteCreateFactory](#) function returns a DirectWrite factory interface pointer via its last parameter, which has [IUnknown](#) type. For such a function, use [com_ptr::put](#), but reinterpret cast that to [IUnknown](#).

C++/WinRT

```
DWriteCreateFactory(  
    DWRITE_FACTORY_TYPE_SHARED,  
    __uuidof(dwriteFactory2),  
    reinterpret_cast<IUnknown**>(dwriteFactory2.put()));
```

Re-seat a winrt::com_ptr

❗ Important

If you have a [winrt::com_ptr](#) that's already seated (its internal raw pointer already has a target) and you want to re-seat it to point to a different object, then you first need to assign `nullptr` to it—as shown in the code example below. If you don't, then an already-seated [com_ptr](#) will draw the issue to your attention (when you call [com_ptr::put](#) or [com_ptr::put_void](#)) by asserting that its internal pointer is not null.

C++/WinRT

```
winrt::com_ptr<ID2D1SolidColorBrush> brush;  
...  
    brush.put()  
...  
brush = nullptr; // Important because we're about to re-seat  
target->CreateSolidColorBrush(  
    color_orange,  
    D2D1::BrushProperties(0.8f),  
    brush.put());
```

Handle HRESULT error codes

To check the value of a HRESULT returned from a COM function, and throw an exception in the event that it represents an error code, call [winrt::check_hresult](#).

C++/WinRT

```
winrt::check_hresult(D2D1CreateFactory(
    D2D1_FACTORY_TYPE_SINGLE_THREADED,
    __uuidof(factory),
    options,
    factory.put_void()));
```

COM functions that take a specific interface pointer

You can call the [com_ptr::get](#) function to pass your `com_ptr` to a function that takes a specific interface pointer of the same type.

C++/WinRT

```
... ExampleFunction(
    winrt::com_ptr<ID2D1Factory1> const& factory,
    winrt::com_ptr<IDXGIDevice> const& dxdevice)
{
    ...
    winrt::check_hresult(factory->CreateDevice(dxdevice.get(), ...));
    ...
}
```

COM functions that take an IUnknown interface pointer

You can use [com_ptr::get](#) to pass your `com_ptr` to a function that takes an **IUnknown** interface pointer.

You can use the [winrt::get_unknown](#) free function to return the address of (in other words, a pointer to) the underlying raw **IUnknown interface** of an object of a projected type. You can then pass that address to a function that takes an **IUnknown** interface pointer.

For info about *projected types*, see [Consume APIs with C++/WinRT](#).

For a code example of `get_unknown`, see [winrt::get_unknown](#), or the [Full source code listing of a minimal Direct2D application](#) in this topic.